

Automated Secure Software Delivery Using GitOps on AWS EKS

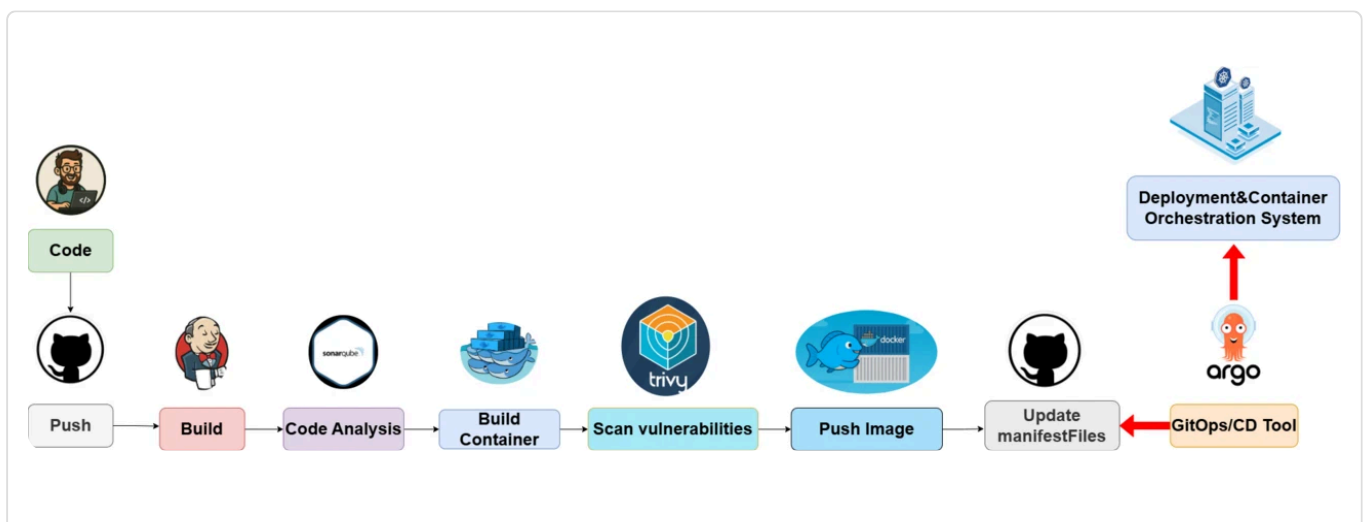
Built Using Git, Jenkins, Maven, Sonarqube, Docker, Trivy, ArgoCD & Kubernetes(EKS)

1. Project Introduction

This document describes a complete DevOps project implementing a CI/CD (Continuous Integration/Continuous Delivery) pipeline for a containerized application. The pipeline automates the process of building, testing, and deploying a Java web application, with a focus on modern DevOps practices such as GitOps and containerization. Key components include a source code repository, a CI server, build automation, static code analysis, containerization, image scanning, a container registry, a GitOps deployment manager, and a Kubernetes cluster for orchestration. The project leverages open-source tools like Jenkins, Maven, SonarQube, Docker, Trivy, Argo CD, and Kubernetes to achieve end-to-end automation and reliability in the software delivery lifecycle.

2. Pipeline Architecture Overview

The entire project is based on the following CI/CD architecture, which illustrates the flow of an application from a developer's commit to a running container in a Kubernetes cluster. Each stage is handled by a specialized tool, creating a powerful and cohesive toolchain.



This diagram illustrates a CI/CD pipeline for containerized applications. The process begins with "Source Code" stored in "GitHub," which is then processed by a "CI/CD Server" like "Jenkins." The code goes through a "Build Stage" using "Maven," followed by "Static Code Analysis" with tools like "SonarQube." Next, the code is used to "Build & scan (Trivy) docker image." The built image is then pushed to "Docker Hub." Subsequently, "CD manifest files repo for Argo CD" are updated. Finally, a "GitOps/CD Tool" like "Argo CD" is used for "Container App Deployment & Orchestration" on "Kubernetes."

Core Components and Workflow:

1. Source Code Management: Developers push code to a GitHub repository.
2. CI Trigger: A webhook from GitHub triggers a pipeline job on the Jenkins CI/CD server.
3. Build & Analyze: Jenkins checks out the code, compiles it using Maven, and runs a static code analysis scan with SonarQube.
4. Containerize & Scan: A Docker image is built from the application code. This image is then scanned for known vulnerabilities using Trivy.
5. Store Artifact: If the scans pass, the validated Docker image is pushed to a container registry, such as Docker Hub.
6. GitOps Trigger: The CI pipeline automatically updates a Kubernetes manifest file in a separate Git repository (the "Config Repo"), pointing to the new Docker image tag.
7. Deploy & Orchestrate: Argo CD, a GitOps tool, detects the change in the Config Repo and automatically synchronizes the state of the Kubernetes cluster to match the desired state defined in the manifests, deploying the new version of the application.



3. Phase 1: Prerequisites and Environment Setup

Before beginning the implementation, it is essential to set up the necessary accounts, infrastructure, and tools. This foundational phase ensures that all components of the pipeline can communicate and operate correctly.

3.1. Required Accounts and Services

- **GitHub Account:** To host the application source code and the Kubernetes configuration repositories. A free account is sufficient for public repositories.
- **Docker Hub Account:** To store and distribute the container images. A free account provides public repositories.
- **Cloud Provider / Kubernetes Environment:** Access to a Kubernetes cluster. Options include:
 - **Local Development:** Gitbash or Visual Studio Code or MobaXterm for local testing.
 - **Cloud-Managed:** Amazon Elastic Kubernetes Service (EKS) for more robust, production-like environments.

3.2. Infrastructure and Tool Installation

A Linux server (e.g., Amazon Linux 2) is required for Jenkins, Docker, and SonarQube setup, with a recommended instance type of "t2.xlarge" or higher. An EKS Cluster is needed to deploy the container application.

3.2.1. Jenkins Server Setup

```
# Update the system
sudo yum update -y

# Add the Jenkins repo
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo yum upgrade -y

# Install Java 17 (required for recent Jenkins versions)
sudo yum install java-17-amazon-corretto -y

# Install Jenkins
sudo yum install jenkins -y

# Enable and start the Jenkins service
sudo systemctl enable jenkins
```

```
sudo systemctl start jenkins
sudo systemctl status jenkins
```

3.2.2. Installation & Configuration of Docker

```
# Install Docker
sudo yum install docker -y
sudo systemctl start docker
sudo systemctl enable docker

# Add jenkins and current user to the docker group to run
docker commands without sudo
sudo usermod -aG docker jenkins
sudo usermod -aG docker ec2-user

# Restart docker and jenkins to apply group changes
sudo systemctl restart docker
sudo systemctl restart jenkins

# (Optional but common) Adjust permissions on the Docker socket
sudo chmod 666 /var/run/docker.sock
```

3.2.3. AWS CLI Configuration

```
# Configure AWS CLI to interact with AWS services
aws configure

# Enter your AWS Access Key ID, Secret Access Key, region,
and output format when prompted.
```

3.2.4. Kubectl and eksctl Installation

```
# Install kubectl
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubec
```

```
rm kubect1
```

```
# Install eksctl
```

```
curl --silent --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl\_\$\(uname -s\)\_amd64.tar.gz" | tar  
sudo mv /tmp/eksctl /usr/local/bin
```

3.2.5. Create EKS Cluster

```
# Create an EKS cluster using eksctl
```

```
eksctl create cluster --name my-app-cluster --nodegroup-name  
my-nodegroup --node-type t3.micro --nodes 2 --managed
```

3.2.6. Install Required Jenkins Plugins

Install necessary plugins in Jenkins:

1. Go to Jenkins Dashboard > Manage Jenkins > Plugins.
2. On the "Available plugins" tab, search for and install: Docker Pipeline, Pipeline: Stage View, Blue Ocean, SonarQube Scanner for Jenkins, and Credentials Binding.

3.2.7. Install and Configure SonarQube as a Docker Container

1. Run SonarQube in a Docker container:

```
docker run -itd --name sonarqube -p 9000:9000 sonarqube:latest
```

2. Check if the SonarQube container is running:

```
docker ps
```

3. Access the SonarQube web interface at <http://<your-server-ip>:9000>.
4. Log in with default credentials (Username: admin, Password: admin) and change the password when prompted.
5. Create a Sonar token for Jenkins: In the SonarQube Dashboard, go to My Account > Security > Generate Tokens.

3.2.8. Generate GitHub Token

Generate a GitHub Personal Access Token (PAT) for Jenkins to access your repositories:

1. Go to GitHub > Settings > Developer settings > Personal access tokens.
2. Generate a new token with the necessary scopes (e.g., repo, admin:repo_hook).

3.2.9. Installation of Argo CD

```
# Create a namespace for Argo CD
kubectl create namespace argocd

# Apply the Argo CD installation manifest
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# To access the Argo CD API server, you can use port-forwarding
kubectl port-forward svc/argocd-server -n argocd 8080:443
```



4. Phase 2: Source Code and Configuration Management

This phase involves setting up the Git repositories that will be the single source of truth for our application and its deployment configuration.

4.1. Application Source Code Repository ("App Repo")

Create a new repository on GitHub for your application. For this project, we'll assume a simple Java application built with Maven.

Recommended Repository Structure:

```
my-app/
├─ src/
│   └─ main/
```

```
|   └─ test/
└─ .github/
|   └─ workflows/      # (Optional for GitHub Actions)
└─ k8s/
|   └─ deployment.yaml # Kubernetes manifest templates
|   └─ service.yaml
└─ Dockerfile          # Instructions to build the container image
└─ Jenkinsfile         # The CI/CD pipeline definition for Jenkins
└─ pom.xml             # Maven project configuration
└─ README.md
```

4.2. Kubernetes Manifest Repository ("Config Repo")

Create a second, separate repository on GitHub. This repository will hold the definitive Kubernetes manifests that Argo CD will use to deploy the application. It is the core of the GitOps workflow.

Recommended Repository Structure:

```
my-app-config/
└─ my-app/
|   └─ deployment.yaml
|   └─ service.yaml
```

Create `deployment.yaml` and `service.yaml` in your repository to define the Kubernetes resources.

`deployment.yaml`:

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  selector:
```

```
matchLabels:
  app: my-app
template:
  metadata:
    labels:
      app: my-app
  spec:
    containers:
      - name: my-app
        image: your-dockerhub-username/your-repo:tag
        # This will be updated by the CI pipeline
        ports:
          - containerPort: 8080
```

service.yaml:

```
---
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
spec:
  type: LoadBalancer
  selector:
    app: my-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

Why two repositories? Separating application code from deployment configuration is a best practice. It provides a clear separation of concerns: developers work in the App Repo, while the CI system and operations team manage the Config Repo. This enhances security and stability.



5. Phase 3: Continuous Integration (CI) Implementation with Jenkins

The CI pipeline is defined as code using a Jenkinsfile. This file lives in the root of the App Repo and orchestrates all the steps from code checkout to pushing the container image.

5.1. The Jenkinsfile: Pipeline as Code

Below is a comprehensive, commented Jenkinsfile that implements the CI stages. This file should be created in the root of your App Repo.

```
pipeline {
    agent any

    tools {
        maven 'maven3' // Assumes a Maven tool configuration name
    }

    environment {
        SONAR_URL = "http://your-sonarqube-ip:9000"
        DOCKER_IMAGE = "your-dockerhub-username/your-repo:${env.BUILD_NUMBER}"
        CONFIG_REPO_URL = "https://github.com/your-github-username/my-app-config.git"
    }

    stages {
        stage('Checkout') {
            steps {
                echo 'Cloning GitHub Repo'
                git branch: 'main', url: 'https://github.com/your-github-username/my-app.git'
            }
        }

        stage('SonarQube Scan') {
            steps {

```

```

        echo 'Scanning project with SonarQube'
        withCredentials([string(credentialsId:
            'sonar-token', variable: 'SONAR_TOKEN')]) {
            sh "mvn clean verify sonar:sonar -Dsonar.host
                url=${SONAR_URL} -Dsonar.login=${SONAR_TOKEN}"
        }
    }
}

stage('Build Artifact') {
    steps {
        echo 'Building JAR/WAR artifact'
        sh 'mvn clean package'
    }
}

stage('Build & Scan Docker Image') {
    steps {
        echo "Building Docker Image: ${DOCKER_IMAGE}"
        sh "docker build -t ${DOCKER_IMAGE} ."
        echo "Scanning Docker Image with Trivy"
        sh "trivy image --exit-code 1 --severity HIGH,
            CRITICAL ${DOCKER_IMAGE}"
    }
}

stage('Push to Docker Hub') {
    steps {
        echo "Pushing Docker Image to Docker Hub"
        withCredentials([usernamePassword(credentialsId:
            'dockerhub-creds', usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS')]) {
            sh "docker login -u ${DOCKER_USER} -p
                ${DOCKER_PASS}"
            sh "docker push ${DOCKER_IMAGE}"
        }
    }
}

stage('Update Deployment File in GitOps Repo') {

```

```

    steps {
        echo "Updating Kubernetes deployment file"
        withCredentials([string(credentialsId:
            'github-token', variable: 'GITHUB_TOKEN')]) {
            sh """
                git clone https://oauth2:\${GITHUB\_TOKEN}@github.com/your-github-username/my-app-cd
                cd my-app-config
                sed -i 's|image: .*|image: ${DOCKER_IMAGE} my-app/deployment.yaml
                git config user.email "jenkins@example.co
                git config user.name "Jenkins CI"
                git add my-app/deployment.yaml
                git commit -m "Update image to
                ${DOCKER_IMAGE} [ci skip]"
                git push
            """
        }
    }
}
}
}
}
}

```

Each stage explanation:

1. Checkout: This stage clones the source code from the GitHub repository. It ensures Jenkins always works with the latest code from the main branch.
2. SonarQube Scan: The source code is analyzed using SonarQube for code quality and security issues. It checks for bugs, vulnerabilities, and code smells before the build continues.
3. Build Artifact: Maven compiles the source code and packages it into a deployable artifact (e.g., JAR/WAR). This verifies that the application builds successfully.
4. Build & Scan Docker Image: A Docker image is created using the Dockerfile. The image is tagged with the Jenkins build number for versioning and then scanned for vulnerabilities with Trivy.
5. Push to Docker Hub: The validated Docker image is pushed to Docker Hub using secured credentials, making it available for deployment.

6. Update Deployment File: The Kubernetes deployment YAML file in the separate GitOps repository is automatically updated with the new Docker image tag. This change is committed and pushed, triggering the GitOps workflow.



6. Phase 4: Continuous Deployment (CD) with Argo CD

With the CI pipeline pushing configuration changes to the Config Repo, the final step is to configure Argo CD to watch this repository and apply the changes to our Kubernetes cluster.

6.1. Creating the Application in Argo CD

You can define an Argo CD application declaratively with a YAML manifest or through the Argo CD UI.

6.1.1. Using the Argo CD UI

1. Log in to your Argo CD UI.
2. Click on + NEW APP.
3. Fill in the application details:
 - Application Name: `my - app`
 - Project: `default`
 - Sync Policy: `Automatic`
 - Check Prune Resources and Self Heal for a fully automated GitOps experience.
4. Configure the Source:
 - Repository URL: The URL of your Config Repo (e.g., `https://github.com/your-github-username/my-app-config.git`).
 - Revision: `HEAD` (or your main branch name).
 - Path: The path to the manifests for this specific application (e.g., `my - app`).
5. Configure the Destination:

- Cluster URL: `https://kubernetes.default.svc` (for in-cluster deployment).
- Namespace: The Kubernetes namespace where you want to deploy the app (e.g., `production`). Ensure this namespace exists.

6. Click the CREATE button.

Deployment Complete! Once the Argo CD application is created and synced, it will immediately fetch the manifests from your Config Repo and deploy the application. From this point on, any commit to the main branch of your App Repo will trigger the entire CI/CD pipeline, resulting in a new, validated image being automatically deployed to Kubernetes within minutes.

7. Project Conclusion and Next Steps

This project plan has detailed the end-to-end implementation of a modern, GitOps-driven CI/CD pipeline. By following these phases, an organization can establish a highly automated, secure, and efficient workflow for developing and deploying containerized applications on Kubernetes.

Potential Enhancements and Further Steps:

- **Advanced Branching and Environments:** Extend the pipeline to support multiple environments (e.g., staging, production) based on Git branches. Use Kustomize or Helm in the Config Repo to manage environment-specific configurations.
- **Secrets Management:** Integrate a dedicated secrets management tool like HashiCorp Vault or the External Secrets Operator instead of storing sensitive information directly in Jenkins or Git.
- **Dynamic Test Environments:** Configure the pipeline to automatically spin up ephemeral preview environments for each pull request, allowing for better testing and review before merging to the main branch.
- **Monitoring and Observability:** Deploy monitoring tools like Prometheus and Grafana, and logging solutions like the EFK stack (Elasticsearch, Fluentd, Kibana) to gain insights into the deployed application's performance and health.
- **Cost Optimization:** Implement tools like Kubecost to monitor and optimize the resource utilization and cost of the Kubernetes cluster.